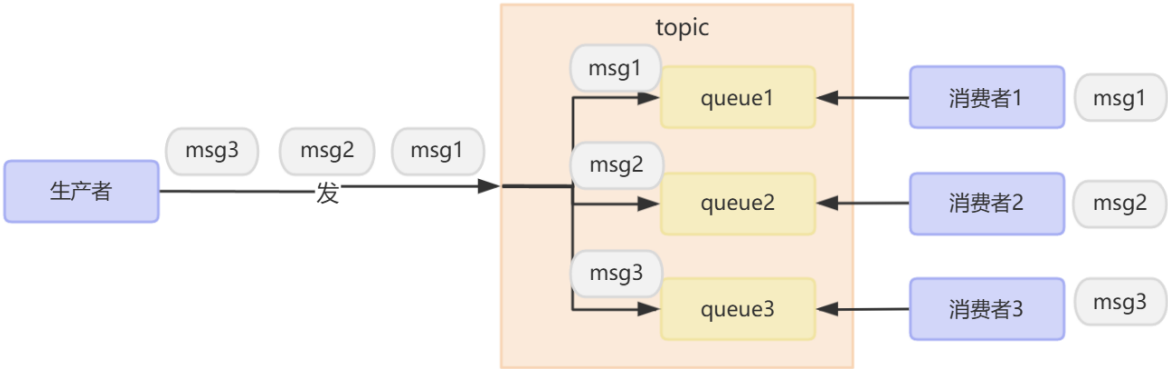
消息顺序性

为什么消息会乱序

要保证消息的顺序性，就得先知道消息为啥会乱序?

同一个topic下, 有多个队列, 或者说多个分区. 消息1 2 3被发送到了同一个topic下的不同队列中, 然后不同的消费者去消费, 这个时候消息就是乱序消费的.

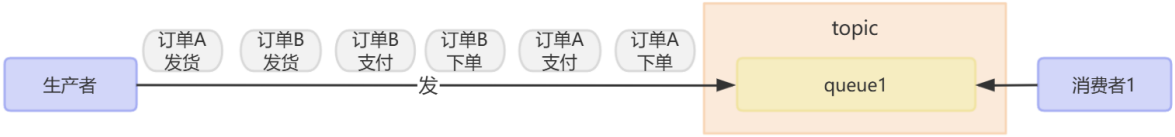


保证消息有序的本质

无论是kafka, 还是rabbitmq还是rocketmq, 单个队列或者说单个分区内的消息都是有序的, 跨队列或者跨分区不能保证消息有序. 所以我们要保证消息有序就两个点, 第一, 自定义消息路由算法, 把消息路由到同一个队列中, 第二, 单个消费者串行消费消息.

全局有序

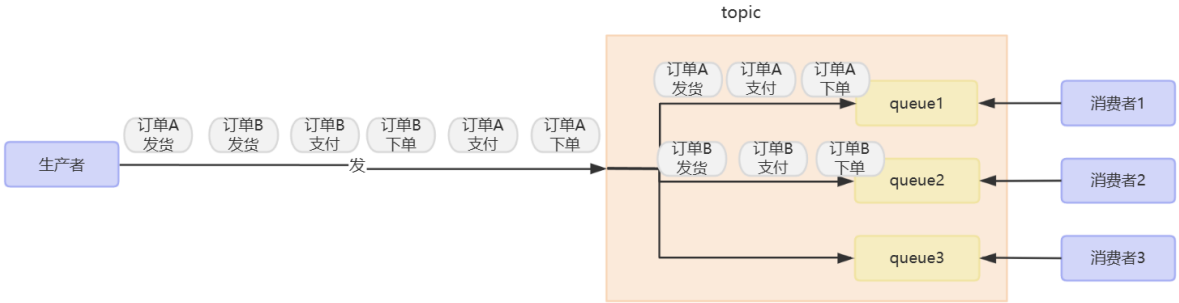
有序分为全局有序和局部有序. 全局有序就是说所有的消息都是有序的, 比如说一个系统中有很多个订单, 这些订单的所有消息都要求有序. 要保证全局有序就只能有一个队列, 所有消息都发到这个队列, 然后一个消费者串行消费. 只用单个队列处理明显性能很差, 问题也很大.



局部有序

其实全局有序这个需求就有问题, 一般我们需要保证的是局部有序, 就是同一个订单, 下单, 支付, 发货, 这是有序的就够了. 同一个订单的消息需要保证顺序, 不同订单之间的顺序性不需要保证, 这就是局部有序.

所以, 一般情况下, 我们会把业务id, 比如根据订单id, 把相同订单的消息路由到同一个队列中, 不同的订单消息就可以到不同的队列, 然后, 一个队列一个消费者消费. 这样既利用mq多队列的特性, 提高了性能, 又保证了相同订单消息的顺序性.



所以局部有序的核心就是把相同id的消息路由到同一个队列，一个队列一个消费者.

消费者多线程并行消费

那如果消费者使用线程池并行消费, 这又会造成乱序问题, 应该怎么办呢?

消费者可以维护几个内存中的阻塞队列, 然后同样的, 把同一个订单的消息放到同一个队列中, 一个队列对应一个线程进行并行消费.

顺序消费的topic能扩容吗

那么问题来了, 顺序消费的topic, 里面的分区数或者队列数能扩容吗?

显然不能轻易扩容或者缩容, 顺序消息的topic, 它的分区数或者队列数必须固定. 因为扩容会导致队列数量变化, 这个key被路由到其他分区或者队列了. 然后就是扩缩容, 或者说某个分区挂了就会触发Rebalance, 就是重新平衡, 那某个分区的消费者可能会从A变到B。A已拉取但尚未消费的若干消息，会被 B 从更早或不同的位置重新拉取处理. 这也可能会导致乱序问题或者幂等性问题. 所以顺序消息一定要做好幂等性保证.

顺序消息如何重试

那么问题又来了, 一般情况下, 消息消费失败了会放到重试队列中重试, 那么顺序消息应该怎么重试呢?

顺序消息消费失败，放到重试队列的话，顺序性就无法保证了。所以顺序消息只能本地重试。本地重试在遇到rebalance时，消息可能会被其他消费者处理, 导致乱序问题或者幂等性问题, 和刚才说的rebalance情况差不多.

对账与补偿

mq本质就是一个高性能但是不太可靠的中间件, 不应该完全依赖mq去保证顺序性和可靠性, 在极端情况下是有可能出问题的, 所以一定要做好兜底措施. 比如说通过定时任务做好数据对账, 补偿, 保证数据的最终一致性.